



AquaGrid

An AI that forecasts an island's water demand, schedules desalination against the solar forecast, harvests the rain, and catches failures before they get expensive.

Rishik · Neekin · Srihan · Ednit · Gowtham

DreamHacks 2026 · Track 2 – AI, Automation & Logic

Water and energy are the same problem

Fresh water comes from rain when it falls, and from a desalination plant when it does not.

That plant is the **largest electrical load on the island.**

Its power is solar when the sun is up, diesel when it is not.

And the diesel arrives **by barge, every few weeks** — whether or not you are running low.

So every hour is a decision

**Noon, clear
sky**

nearly free

02:00

imported diesel

The tank

holds a day or two

The decision **can** be deferred — if you know what tomorrow looks like.

Almost nowhere is it made

It is delegated to a **wall timer**, usually set to overnight hours inherited from mainland off-peak electricity tariffs.

On a solar island that is exactly backwards. Those are the hours with no sun.

Tuvalu makes it concrete. The freshwater lens is thin and salt-contaminated, so the island runs on rain. In **2011 a drought left Funafuti with days of water** and a declared state of emergency. The plant was not too small — nobody could see far enough ahead to act early.

One forecast, shared by everything

01

Predict

Water demand, 48 h ahead

02

Decide

Eight rules, 12 h lookahead

03

Explain

A reason for every hour

04

Detect

Failures nobody would notice

Predict demand once, and four separate blind spots become one solved problem.

The model is real, and tested honestly

45.3_{L/h}

mean absolute error

0.987

R^2

89%

better than predicting the average

Chronological split — the last 60 days held out, so it is tested on the future, never on shuffled rows.

78% less diesel

STRATEGY	DIESEL	COST	CO ₂
AquaGrid	15.3 L	\$25	41 kg
Fixed timer 01–05	70.0 L	\$112	188 kg
Reactive < 80%	79.2 L	\$127	212 kg

Same water, **fewer** pump hours. It just chooses which hours.

The number that matters more

The fuel tank holds **300 L**. The next barge is **12 days** away.

STRATEGY	BURN	FUEL LASTS	OUTCOME
AquaGrid	8 L/day	39 days	holds
Fixed timer	35 L/day	8.6 days	dry -3.4 d
Reactive	40 L/day	7.6 days	dry -4.4 d

Burning too much diesel and running out of diesel are **different failures**.

Four layers. The laptop trains once.

LAYER 1 · PYTHON, ONCE	Train & export	history → RandomForest → lookup table
LAYER 2 · ONE FILE	forecast.json	the model, plus a seed forecast
LAYER 3 · PURE JS	Decide & detect	no React imports → testable with node
LAYER 4 · REACT NATIVE	Present	SVG charts, optional LLM

After training, the laptop is out of the loop. **The phone fetches its own forecast.**

We ship the model, not just its predictions

Every feature is categorical except temperature — so the trained forest is evaluated over its **entire input grid** and exported as a lookup table.

15,120

cells — $2 \times 7 \times 24 \times 45$

5.5 L/h

fidelity loss vs the model's own
45.3

0.011 ms

72 predictions, on the phone

A real forecaster running on-device — for islands the laptop never saw.

No backend, on purpose

An app that needs a server to decide whether to run a pump fails **exactly when an island is cut off** — which is exactly when it matters.

Two AI systems. Only one decides.

RandomForest

Trained by us. Makes

every

decision — every pump hour, every alert.

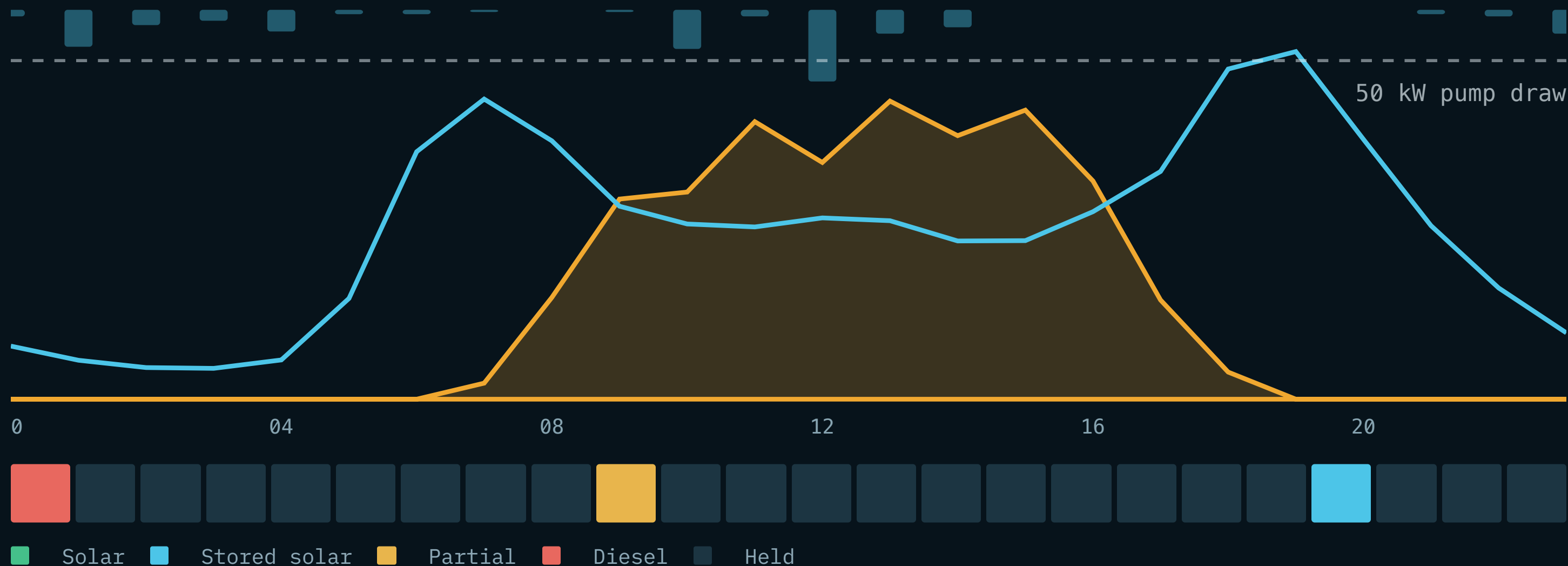
GPT-OSS 120B

Open-weight, optional. Makes

none

. Remove it and every number is unchanged.

One day, as the scheduler sees it



Eight rules. First match wins.

0 Storm preparation	1 Emergency	2 Rain hold	3 Free solar
4 Stored solar	5 Pre-emptive hybrid	6 Reluctant diesel	7 Hold

The highlighted three act on information the plant **does not yet have**. Rule 5 spends a little diesel now to avoid a lot tonight.

Rain, battery, and four detectors

24%

of all water came from rain – the pump holds when a shower is forecast

66 kWh

midday solar carried into the evening peak

304 kWh

curtailed – the honest ceiling on a bigger battery

Burst pipe

caught in 2 h

Dirty panels

13% below irradiance

Fouling membranes

clean due in 40 days

Fuel

dry before the barge

All four the same idea: compare what the plant **does** against what the forecast says it **should**.

Demo

1 Current decision — read the reason line	2 Chart & timeline — green free, cyan stored, amber deliberate
3 Savings, then fuel vs barge	4 Simulate burst pipe — alert fires
5 Simulate cyclone — objective flips	6 Change island — everything recomputes on-device
7 Ask the operator, then a follow-up	8 Airplane mode. Nothing stops working.

The code

755

lines in the decision layer

0

React imports in it – so it tests
without a phone

node
src/logic/test.js

runs all three strategies, the leak,
and the table fidelity

APIs — two of the three need no key at all

Open-Meteo Forecast

solar, temp, rain, wind · no key

Open-Meteo Geocoding

name → coords + population · no key

Groq

open-weight LLM · optional



Everything is computed, not asserted.

Three schedulers over the same forecast and the same physics. A model tested on the days that came after. A leak detector given a real injected fault.

Reykjavík sits in the results at 23% next to Apia at 80%, because a smaller honest number beats a larger one we cannot defend.

github.com/sbongula/aquagrid

sbongula.github.io/aquagrid